SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1401 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.(BL3)*

Assessment Tool Weightage:10%

Duration :60 Mins
On Class
QUESTIONS: 50 **Total Marks:50**

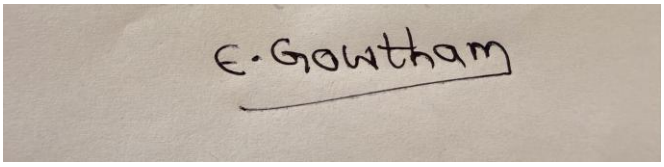
Annexure A

MOCK INTERVIEW

Code of Conduct:

I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

MOCK INTERVIEW

Q1. What is the primary goal of code generation, and how does it affect program performance?

Q2. How do target machine characteristics influence code generation?

Q3. Explain the role of intermediate code in optimization and portability.

Q4. Why is machine-independent optimization preferred before machine-dependent optimization?

Q5. What are the major sources of inefficiency in unoptimized code?

Q6. How does common subexpression elimination reduce computation time?

Q7. Can common subexpression elimination ever be unsafe? Justify your answer.

Q8. Explain how constant folding and constant propagation work together.

Q9. How do you detect and eliminate dead code in a program?

Q10. What are the limitations of peephole optimization?

Q11. Explain different peephole optimization techniques with examples.

Q12. How does strength reduction improve execution efficiency?

Q13. What is the role of algebraic simplification in optimization?

Q14. How are redundant instructions identified and removed?

Q15. Why are basic blocks important in optimization?

Q16. What conditions must a sequence of statements satisfy to form a basic block?

Q17. Explain the process of constructing a control flow graph (CFG).

Q18. How does a CFG help in program analysis and optimization?

Q19. Compare local optimization vs global optimization with examples.

Q20. What are the challenges in global data flow analysis?

Q21. How does live variable analysis help in optimization?

Q22. Explain the concept of reaching definitions with a practical example.

Q23. What is available expression analysis, and where is it used?

Q24. How do data dependencies affect instruction execution?

Q25. What are control dependencies, and why are they important?

Q26. What are the key design issues in a code generator?

Q27. How is instruction selection performed in compilers?

Q28. What factors affect register allocation strategies?

Q29. When does register spilling occur, and how is it handled?

Q30. How does next-use information guide register allocation?

Q31. What is the role of a simple code generator?

Q32. What assumptions are made in designing a simple code generator?

Q33. How are register descriptors and address descriptors used?

Q34. Explain the structure and purpose of a target machine.

Q35. Compare register-based and stack-based machines in code generation.

Q36. What are different addressing modes, and how do they affect code generation?

Q37. How does instruction scheduling improve performance?

Q38. What is runtime storage management, and why is it necessary?

Q39. Compare stack allocation and heap allocation.

Q40. What is an activation record, and what are its components?

Q41. How is a DAG constructed for a basic block?

Q42. How does a DAG help in common subexpression elimination?

Q43. How does DAG representation assist in dead code elimination?

Q44. What are the advantages of using DAG over syntax trees?

Q45. What is loop optimization, and why is it important?

Q46. Explain loop-invariant code motion with an example.

Q47. What is induction variable elimination, and how does it help?

Q48. How does loop unrolling improve performance?

Q49. What is code motion across basic blocks?

Q50. How do modern compilers combine multiple optimization techniques effectively?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

MOCK INTERVIEW – CODE OPTIMIZATION & CODE GENERATION

Q1. What is the primary goal of code generation, and how does it affect program performance?

The primary goal of code generation is to convert **intermediate code into efficient target-machine code**. Good code generation reduces execution time, memory usage, and the number of instructions, improving overall program performance.

Q2. How do target machine characteristics influence code generation?

The target machine determines:

- Available registers
- Instruction set
- Addressing modes
- Memory organization
- Word size
- Cost of instructions

The compiler uses these characteristics to generate efficient machine code.

Q3. Explain the role of intermediate code in optimization and portability.

Intermediate code provides a **machine-independent representation** between source code and machine code.

It allows:

- Optimization before target-specific code generation.
- The same front end to support multiple target machines.
- Easier implementation of compiler optimizations.

Q4. Why is machine-independent optimization preferred before machine-dependent optimization?

Machine-independent optimization can improve the program **regardless of the target hardware**. After that, machine-dependent optimization can take advantage of specific processor features such as registers and special instructions.

Q5. What are the major sources of inefficiency in unoptimized code?

Common sources include:

- Repeated calculations
- Dead code
- Unnecessary memory accesses
- Inefficient loops
- Excessive instructions
- Unnecessary register transfers
- Poor register usage

Q6. How does common subexpression elimination reduce computation time?
It identifies an expression that is calculated multiple times and calculates it only once.
Example:
a = b + c
d = b + c
Instead of calculating b + c twice, the result of the first calculation can be reused.

Q7. Can common subexpression elimination ever be unsafe? Justify your answer.
Yes. It can be unsafe when the expression may produce a different result because its operands or program state can change.
For example, expressions involving **function calls, volatile memory, or side effects** may not be safely reused.

Q8. Explain how constant folding and constant propagation work together.
Constant propagation replaces variables with their known constant values, while **constant folding** evaluates the resulting constant expression.
Example:
a = 10
b = a + 5
Constant propagation gives:
b = 10 + 5
Constant folding then gives:
b = 15

Q9. How do you detect and eliminate dead code in a program?
Dead code is code whose result is never used or whose execution cannot affect the program's observable behavior.
Compilers use techniques such as **liveness analysis and control-flow analysis** to identify it and then remove it.

Q10. What are the limitations of peephole optimization?
Peephole optimization:

- Examines only a small sequence of instructions.
- May miss optimization opportunities across basic blocks.
- Has limited knowledge of the whole program.
- Depends on the target machine.
- Cannot perform complex global optimizations effectively.

Q11. Explain different peephole optimization techniques with examples.
Major techniques include:
1. Redundant instruction elimination

MOV R1, R2
MOV R1, R2
Remove the duplicate instruction.

2. Algebraic simplification

ADD R1, 0
can be removed.

3. Strength reduction

MUL R1, 2
can sometimes be replaced by a shift.

4. Constant folding

MOV R1, 5
ADD R1, 3
can become:
MOV R1, 8

5. Redundant load/store elimination

Unnecessary memory operations are removed when the required value is already available.

Q12. How does strength reduction improve execution efficiency?

Strength reduction replaces an expensive operation with a cheaper equivalent operation.
For example:
x = y * 2
can sometimes be implemented using a shift:
x = y << 1
This can reduce execution cost on suitable target machines.

Q13. What is the role of algebraic simplification in optimization?

Algebraic simplification reduces unnecessary operations using mathematical identities.
Examples:
x + 0 → x
x * 1 → x
x * 0 → 0
This reduces instruction count and improves performance.

Q14. How are redundant instructions identified and removed?

The compiler analyzes whether an instruction produces a result that is already available or is never used. If removing the instruction does not change program behavior, it can be eliminated.

Q15. Why are basic blocks important in optimization?

A basic block has a **single entry and single exit**, making it easier for the compiler to analyze computations and dependencies.

They are especially useful for:

- Local optimization
- DAG construction
- Data-flow analysis
- CFG construction

Q16. What conditions must a sequence of statements satisfy to form a basic block?

A basic block must:

1. Have **one entry point**.
2. Have **one exit point**.
3. Have no branching into its middle.
4. Have control flow entering only at its first statement.

Q17. Explain the process of constructing a control flow graph (CFG).

The general process is:

1. Identify leaders.
2. Divide the program into basic blocks.
3. Create one CFG node for each basic block.
4. Add directed edges according to possible control flow.
5. Connect blocks following conditional and unconditional branches.

Q18. How does a CFG help in program analysis and optimization?

A CFG shows all possible execution paths. It helps the compiler perform:

- Dead code elimination
- Loop detection
- Data-flow analysis
- Reachability analysis
- Global optimization

Q19. Compare local optimization vs global optimization with examples.

Local Optimization	Global Optimization
Works within one basic block	Works across multiple basic blocks
Smaller scope	Larger scope
Easier to perform	More complex
Example: constant folding	Example: global CSE

Example: Removing x + 0 within one block is local optimization. Reusing a calculation across different blocks can be global optimization.

Q20. What are the challenges in global data flow analysis?

Major challenges include:

- Multiple execution paths
- Loops
- Conditional branches
- Large programs
- Maintaining accurate information at block boundaries
- Ensuring optimization does not change program behavior

Q21. How does live variable analysis help in optimization?
It determines whether a variable's current value will be used later.
If a value is **not live**, the instruction producing it may be removed, and its register can potentially be reused.

Q22. Explain the concept of reaching definitions with a practical example.
A definition is an assignment that gives a value to a variable.
Example:
a = 10
...
a = 20
...
b = a + 5
At b = a + 5, the definition a = 20 reaches that statement, while a = 10 does not if it has been overwritten on every path.
Reaching definitions help determine **which assignments may provide a value to a use**.

Q23. What is available expression analysis, and where is it used?
Available expression analysis determines which expressions have already been computed and whose operands have not changed.
It is mainly used for **common subexpression elimination**.

Q24. How do data dependencies affect instruction execution?
A data dependency means one instruction needs the result of another instruction.
Example:
a = b + c
d = a + 5
The second instruction must wait for a to be produced. Dependencies therefore affect **instruction ordering and scheduling**.

Q25. What are control dependencies, and why are they important?
Control dependency occurs when execution of a statement depends on a condition or branch.
Example:
if (x > 0)

y = 10;

The execution of y = 10 depends on the condition. Control dependencies are important for maintaining correct program behavior during optimization.

Code Generator

Q26. What are the key design issues in a code generator?

Important issues include:

- Instruction selection
- Register allocation
- Register assignment
- Addressing modes
- Instruction ordering
- Memory management
- Evaluation order
- Target-machine constraints

Q27. How is instruction selection performed in compilers?

The compiler maps intermediate-code operations to suitable target-machine instructions.
It considers:

- Instruction cost
- Available instructions
- Addressing modes
- Register availability
- Target architecture

The goal is to generate efficient machine code.

Q28. What factors affect register allocation strategies?

Factors include:

- Number of available registers
- Frequency of variable use
- Variable lifetime
- Next-use information
- Register classes
- Instruction requirements
- Cost of spilling to memory

Q29. When does register spilling occur, and how is it handled?

Register spilling occurs when **more values are needed than available registers**.
The compiler stores some values in memory temporarily and loads them back when required.

Q30. How does next-use information guide register allocation?

Next-use information tells the compiler **when a variable will be needed again**.
If a register is required for another value, the compiler can choose a variable whose next use is far away—or which has no future use—to free that register.

Q31. What is the role of a simple code generator?
A simple code generator converts intermediate code, such as **three-address code**, into target-machine instructions using straightforward rules.
Its purpose is to demonstrate the basic process of code generation without complex optimization.

Q32. What assumptions are made in designing a simple code generator?
Typical assumptions are:

- A fixed number of registers.
- Simple target instructions.
- Simple addressing modes.
- Intermediate code is already available.
- Basic register and address information is maintained.

Q33. How are register descriptors and address descriptors used?
Register descriptor: tells which values are currently stored in each register.
Address descriptor: tells where the current value of a variable can be found, such as in a register or memory.
Together, they help the code generator avoid unnecessary loads and stores.

Q34. Explain the structure and purpose of a target machine.
A target machine consists of components such as:

- CPU/registers
- Arithmetic and logic unit
- Memory
- Instruction set
- Addressing modes

The code generator uses these features to produce machine-specific code.

Q35. Compare register-based and stack-based machines in code generation.

Register-Based	Stack-Based
Uses CPU registers	Uses an operand stack
Usually requires register allocation	Less explicit register allocation
Can provide efficient access to frequently used values	Operations often use stack push/pop
Common in modern CPUs	Common in virtual machines

Q36. What are different addressing modes, and how do they affect code generation?
Common addressing modes include:

- **Immediate:** operand is directly given.
- **Register:** operand is in a register.
- **Direct:** address of operand is specified.
- **Indirect:** register/memory contains the address.
- **Indexed:** address is calculated using a base plus index.

The compiler chooses an appropriate addressing mode to reduce instructions and memory accesses.

Q37. How does instruction scheduling improve performance?
Instruction scheduling rearranges independent instructions to reduce processor stalls and improve utilization of CPU resources.
It must preserve all required data and control dependencies.

Q38. What is runtime storage management, and why is it necessary?
Runtime storage management controls how memory is allocated and released during program execution.
It is necessary for:

- Local variables
- Function calls
- Dynamic data
- Temporary values
- Recursion

Q39. Compare stack allocation and heap allocation.

Stack Allocation	Heap Allocation
Used for function calls and local data	Used for dynamically allocated data
Automatically managed	Usually explicitly or garbage-collector managed
Fast	Generally more complex
Follows LIFO behavior	Flexible allocation and lifetime

Q40. What is an activation record, and what are its components?
An activation record is a block of memory associated with a **function call**.
It may contain:

- Parameters
- Local variables
- Return address
- Temporary values
- Saved registers

- Control information
- Links to other activation records

DAG and Loop Optimization

Q41. How is a DAG constructed for a basic block?

The steps are:

1. Create leaf nodes for variables/constants.
2. Create an interior node for each operation.
3. Connect operand nodes to the operation node.
4. Assign variables as labels.
5. Reuse an existing node when the same expression already exists.

This exposes common computations.

Q42. How does a DAG help in common subexpression elimination?

Suppose:

a = b + c

d = b + c

A DAG creates only **one node for b + c** and both assignments can use that node. Therefore, the expression needs to be evaluated only once.

Q43. How does DAG representation assist in dead code elimination?

The DAG shows which computed values are actually required. If a node does not contribute to any required result or live variable, its computation may be removed.

Q44. What are the advantages of using DAG over syntax trees?

DAG:

- Shares common expressions.
- Avoids duplicate computation.
- Represents dependencies clearly.
- Helps eliminate redundant computations.
- Helps identify dead code.

A syntax tree normally represents each occurrence separately, so common subexpressions may be duplicated.

Q45. What is loop optimization, and why is it important?

Loop optimization improves the performance of frequently executed loops by reducing unnecessary operations.

Since loops can execute hundreds or millions of times, even a small improvement inside a loop can significantly improve overall performance.

Q46. Explain loop-invariant code motion with an example.

Loop-invariant code motion moves a computation outside a loop when its value **does not change during loop execution**.
Before:

```
for (i = 0; i < n; i++) {
   x = a * b;
   c[i] = x + i;
}
```

If a and b do not change inside the loop:

```
x = a * b;
for (i = 0; i < n; i++) {
   c[i] = x + i;
}
```

The multiplication is performed only once instead of once per iteration.

Q47. What is induction variable elimination, and how does it help?
An induction variable changes by a constant amount during each loop iteration.
The compiler may eliminate unnecessary induction variables by deriving their values from another existing variable. This reduces instructions executed inside the loop.

Q48. How does loop unrolling improve performance?
Loop unrolling reduces loop-control overhead by performing multiple iterations in one loop body.
Original:

```
for (i = 0; i < 4; i++)
   a[i] = a[i] + 1;
```

Conceptually unrolled:

```
a[0] = a[0] + 1;
a[1] = a[1] + 1;
a[2] = a[2] + 1;
a[3] = a[3] + 1;
```

It can reduce branch and loop-counter overhead, though it may increase code size.

Q49. What is code motion across basic blocks?
Code motion across basic blocks means moving a computation from one block to another when it is safe to do so and doing so improves performance.
A common example is moving a **loop-invariant computation** outside a loop.

Q50. How do modern compilers combine multiple optimization techniques effectively?
Modern compilers apply multiple optimization passes in stages.
For example:

Constant propagation → constant folding → dead code elimination → common subexpression elimination → loop optimization → register allocation → machine-dependent optimization

One optimization can create new opportunities for another. Therefore, compilers often perform several passes until the code reaches a good optimized form.